
Software Requirements Specification

for

PredictiX : AI-Powered Predictive Maintenance & Smart Ticket Categorization for Asset Management

V 1.0

Prepared by AI Group 02

Team NeuroMinds

**Department of Computational Mathematics
Faculty of Information Technology
University of Moratuwa**

12/02/2026

Table of Contents

1. Introduction.....	5
1.1 Purpose.....	5
1.2 Document Conventions.....	5
1.3 Intended Audience and Reading Suggestions.....	6
1.4 Product Scope.....	6
1.5 References.....	10
2. Overall Description.....	12
2.1 Product Perspective.....	12
2.2 Product Functions.....	12
2.3 User Classes and Characteristics.....	14
2.4 Operating Environment.....	14
2.5 Design and Implementation Constraints.....	16
2.6 User Documentation.....	17
2.7 Assumptions and Dependencies.....	18
3. External Interface Requirements.....	20
3.1 User Interfaces.....	20
3.2 Hardware Interfaces.....	25
3.3 Software Interfaces.....	26
3.4 Communication Interfaces.....	30
4. System Features.....	32
4.1 Feature 1: User Authentication and Role Management.....	32
4.2 Feature 2: Asset Management.....	33
4.3 Feature 3: Predictive Maintenance and Health Scoring.....	33
4.4 Feature 4: AI-Based Maintenance Cost Estimation.....	34
4.5 Feature 5: Maintenance Ticket Management.....	35
4.6 Feature 6: Dashboards and Data Visualization.....	35
4.7 Feature 7: AI-Generated Reports (RAG Based).....	36
4.8 Feature 8: AI-Powered Chatbot.....	36
4.9 Feature 9: Notifications and Alerts.....	37
4.10 Feature 10: Security, Scalability, and Performance.....	37
5. Other Non Functional Requirements.....	38
5.1 Performance Requirements.....	38
5.2 Safety Requirements.....	38
5.3 Security Requirements.....	38
5.4 Software Quality Attributes.....	39
5.5 Business Rules.....	39
6. Other Requirements.....	41
6.1 Database Requirements.....	41

6.2 Internationalization Requirements..... 41

Appendix A: Glossary..... 42

Appendix B: Analysis Models.....44

Appendix C: To Be Determined List.....47

LIST OF FIGURES

Figure 1 - ER Diagram

Figure 2 - Use case diagram

Figure 3 - Class Diagram

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

The purpose of this System Requirements Specification (SRS) document is to define in detail the functional and non-functional requirements of the PredictiX system, an AI-Powered predictive maintenance & smart ticket categorization for asset management application. This document serves as a formal reference for developers, project supervisors, and stakeholders, ensuring a shared understanding of the system's objectives, scope, constraints, and expected behavior.

PredictiX is designed to enhance warehouse asset management through the integration of predictive analytics, intelligent ticket handling, and data driven reporting. The system provides role based access control to ensure secure interaction between administrators and users, enabling controlled management of assets, maintenance records, operational logs, and system generated insights.

A key objective of the system is to support predictive maintenance by analyzing asset related data, including maintenance logs and sensor readings, to estimate asset health scores, predict failure probabilities, and recommend maintenance timelines. By incorporating machine learning models and structured data analysis, the system aims to reduce unplanned downtime, estimate maintenance costs and optimize maintenance planning within warehouse environments.

In addition to predictive capabilities, PredictiX integrates intelligent ticket management features. These include automated ticket categorization, priority prediction, and summarization to improve maintenance workflow efficiency and transparency. The system also generates comprehensive asset level and warehouse-level reports using Retrieval-Augmented Generation (RAG), ensuring that generated insights are grounded in stored operational data.

This SRS document establishes a clear foundation for the system's design and implementation. It specifies system functionality, performance expectations, user roles, data handling requirements, and integration constraints. By formally documenting these requirements, this specification supports systematic development, validation, and evaluation of the PredictiX platform while ensuring alignment with academic and industrial standards.

1.2 Document Conventions

- High Priority: Must have features.
- Medium Priority: Important but not critical.
- Low Priority: Nice to have features.
- Formatting: Requirements are numbered (e.g., REQ-1, REQ-2)

1.3 Intended Audience and Reading Suggestions

This document is intended for individuals and groups involved in the design, development, evaluation, deployment, and operational use of the PredictiX system. The primary audience includes software developers, data scientists, system architects, project managers, testers, database administrators, and warehouse operations managers.

Warehouse administrators and end users are encouraged to review the overall system description and user role specifications to understand system capabilities, access privileges, and operational workflows.

The document is organized in a structured manner, beginning with an introduction and overall system description, followed by detailed functional requirements, external interface requirements, and non-functional requirements. Subsequent sections address data handling, security considerations, system constraints, and future enhancement possibilities. Appendices may include supporting materials such as a glossary of technical terms, conceptual models, and design references.

This structured organization allows stakeholders to navigate the document efficiently and access information relevant to their roles while maintaining a comprehensive understanding of the PredictiX system.

1.4 Product Scope

The predictive maintenance and Smart ticket categorization for asset management is a transformative initiative designed to improve operational efficiency, reducing equipment downtime, and supporting data-driven maintenance decisions. The system provides real time visibility into asset conditions, automates maintenance ticket handling, predicts potential failures, and generates intelligent reports using AI models. It introduces a suite of advanced functionalities, each strategically crafted to enhance the efficiency of asset conditions and ticket creating . The core components within the project scope are as follows:

1. Role-Based Warehouse Asset Management

- The system provides controlled access through defined user roles, ensuring secure interaction with warehouse assets, maintenance records, and reports.
- Administrators are granted full privileges to manage assets, monitor system analytics, and oversee maintenance operations.
- Standard users are permitted to view assigned assets, create maintenance tickets, and access relevant reports based on authorization levels.

2. Asset Lifecycle and Maintenance Record Management

- The system enables the creation, updating, searching, and deletion of asset records.
- Each asset profile includes descriptive information, operational status, maintenance history, and

associated logs.

- Maintenance schedules and historical service data are stored to support both operational tracking and predictive modeling.

3. Predictive Maintenance, Health Scoring, and Failure Forecasting

- PredictiX incorporates machine learning models to analyze maintenance logs and available sensor readings.
- The system calculates asset health scores based on historical performance and operational indicators.
- Failure probabilities and estimated maintenance timelines are generated to support proactive planning and timely interventions.
- Prediction confidence levels are displayed to assist administrators in evaluating model reliability.

4. Prediction Explainability and Insights

- To promote trust and responsible decision making, PredictiX provides explainability for key predictions produced by its AI models.
- The system highlights the main factors contributing to predicted failure risk or reduced health scores, using interpretable feature importance explanations derived from historical and sensor based indicators.

5. Visualization of Asset and Warehouse Health

- The system presents asset performance and risk information through dashboards and visual analytics to support operational decision making.
- Charts and indicators include health score trends, failure risk forecasts, maintenance timelines, ticket distributions, and workload summaries.
- Warehouse level dashboards provide aggregated views of asset health distribution, high risk assets, and overall maintenance demand across the warehouse environment.

6. Smart Ticket Management

- Users can create, update, and monitor maintenance tickets associated with specific assets and maintenance events.
- PredictiX applies automated ticket categorization and priority prediction using text classification models trained on historical ticket descriptions and maintenance patterns.
- Ticket tracking supports full lifecycle visibility, including issue reporting, assignment, work

progression, and closure, ensuring traceable maintenance workflows.

7. Automated Ticket Summarization and Documentation

- The system supports AI based ticket summarization to improve clarity and traceability across maintenance operations.
- Summaries capture essential ticket details such as the reported issue, affected asset, assigned category, predicted priority, actions taken, and current status.
- Ticket update history and resolution documentation are stored to support accountability, audits, and future learning from past incidents.

8. AI Based Report Generation

- PredictiX generates both asset level and warehouse level analytical reports using a Retrieval Augmented Generation (RAG) approach.
- Reports are grounded in stored operational data, maintenance logs, and predictive model outputs.
- Generated reports include health distributions, high risk asset identification, maintenance workload trends, and strategic recommendations.

9. Maintenance Cost Estimation

- The system provides estimated maintenance related costs to support budgeting and operational planning.
- Cost estimates consider historical repair patterns, expected downtime duration, asset criticality, and predicted failure likelihood.
- PredictiX also highlights potential cost savings achievable through proactive interventions, including reduced emergency repair frequency and prevention of asset downtime.

10. Notification and Alert Management

- PredictiX supports automated notifications to ensure timely awareness of critical events and risks.
- Alerts can be triggered for high risk assets, sharp health score drops, predicted failure thresholds, and urgent maintenance recommendations.
- Notifications also cover operational events such as newly created tickets, overdue maintenance actions, supporting faster response and better coordination.

11. Natural Language Chatbot Query Handling

- The system provides a chatbot interface that enables users to interact with warehouse information

using natural language.

- Users may request information such as current asset health status, predicted maintenance dates, failure probabilities, ticket progress updates, or warehouse level risk summaries.
- The chatbot responds using system data and stored operational context, enabling quick access to insights without requiring manual dashboard navigation.

12. Data Management and Storage

- The system securely stores structured asset data, maintenance logs, ticket records, and predictive outputs with appropriate access control.
- Where available, sensor readings are integrated to enhance prediction accuracy and improve health score calculations.
- A vector database is utilized to support contextual retrieval for AI based reporting and chatbot responses, ensuring that generated content remains aligned with stored warehouse real data.

13. User Roles

- Administrators: Responsible for asset registration, user management, monitoring predictive analytics, reviewing reports, and overseeing warehouse level performance.
- Users (Maintenance Staff / Standard Users): Responsible for reporting issues, creating and updating maintenance tickets and monitoring assigned assets.

14. Main Goals of the Project

- To transition warehouse maintenance practices from reactive responses to predictive strategies supported by data driven analysis.
- To centralize asset management, maintenance tracking, intelligent ticket handling, and AI assisted reporting within a single integrated platform.
- To reduce unplanned downtime by identifying failure risks early and improving the timing and prioritization of maintenance work.
- To improve transparency and accountability through traceable ticket workflows, summarized documentation, and explainable AI outputs.
- To provide a scalable foundation that supports future expansion, additional predictive models, and integration with evolving warehouse environments.

15. Benefits of the Project

- Improved reliability of warehouse assets through early detection of potential failures and

Predictive Maintenance & Smart Ticket Categorization for Asset Management

condition-based maintenance planning.

- Reduced maintenance costs by minimizing emergency repairs, lowering downtime related losses, and avoiding unnecessary servicing.
- Enhanced operational decision making through dashboards, predictive insights, explainable outputs, and AI based reports grounded in warehouse data.
- Higher efficiency in ticket management through automated categorization, priority prediction, summarization, and lifecycle tracking.
- Secure and structured data management that supports operational continuity
- A scalable architecture that can be extended to additional warehouses, larger asset inventories, and future enterprise level integrations.

1.5 References

[1]830-1984 - IEEE Guide for Software Requirements Specifications

<https://ieeexplore.ieee.org/document/278253>

[2]FastAPI Documentation - <https://fastapi.tiangolo.com/>

[3]Posgresql Documentation - <https://www.postgresql.org/>

[4]Multimodal Retrieval Augmented Generation

<https://cs231n.stanford.edu/2024/papers/multimodal-retrieval-augmented-generation-for-instruction-manual.pdf>

<https://developer.nvidia.com/blog/an-easy-introduction-to-multimodal-retrieval-augmented-generation/>

[5] Supabase, “Supabase Documentation,” 2024. Available: <https://supabase.com/docs>

[6] PostgreSQL Global Development Group, *PostgreSQL Documentation*, 2024. Available: <https://www.postgresql.org/docs/>

[7] J. Brownlee, *Machine Learning Mastery with Python*. Machine Learning Mastery, 2020.

[8] ISO, *ISO 55000: Asset Management – Overview, Principles and Terminology*, International Organization for Standardization, 2018.

[9] I. Sommerville, *Software Engineering*, 10th ed. Pearson Education, 2016.

[10] Carvalho, T. P., et al. (2019 - *A systematic literature review of machine learning methods applied to*

Predictive Maintenance & Smart Ticket Categorization for Asset Management

predictive maintenance.Computers & Industrial Engineering, 137, 106024.

[11] Chen, T., & Guestrin, C. (2016) - *XGBoost: A scalable tree boosting system*, Proceedings of the 22nd ACM SIGKDD Conference.

[12] Lundberg, S. M., & Lee, S.-I. (2017) - *A unified approach to interpreting model predictions*,Advances in Neural Information Processing Systems (NeurIPS).

[13] Arrieta, A. B., et al. (2020) - *Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges*.Information Fusion, 58, 82–115.

[14] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019) - *BERT: Pre-training of deep bidirectional transformers for language understanding*.NAACL-HLT.

[15] Sanh, V., et al. (2019) - *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*.arXiv:1910.01108.

[16] Lewis, M., et al. (2020) - *BART: Denoising sequence-to-sequence pre-training for natural language generation*. ACL 2020.

[17] Jones, M., Bradley, J., & Sakimura, N. (2015). - *JSON Web Token (JWT)* - IETF RFC 7519.

2. Overall Description

2.1 Product Perspective

Modern warehouse operations depend heavily on the continuous availability and reliability of physical assets such as conveyors, forklifts, sensors, and automated systems. Unexpected equipment failures can lead to operational delays, increased maintenance costs, and safety risks. Traditional reactive maintenance approaches are no longer sufficient in data driven environments.

This project introduces an AI powered Warehouse Asset Management and Predictive Maintenance System designed to monitor asset health, predict potential failures, and maintenance workflows. The system integrates asset tracking, predictive analytics, maintenance ticketing, and warehouse level insights into a unified platform. By combining real time data visualization with intelligent predictions, the solution supports proactive decision making for both administrators and operational users.

The proposed system focuses on improving asset reliability, reducing downtime, and enhancing maintenance efficiency through automation and data driven insights. Role based access, interactive dashboards, and an integrated chatbot further enhance usability, making the platform suitable for real world warehouse management scenarios.

2.2 Product Functions

- **Asset Registration and Management**

The system allows administrators to create, update, and delete warehouse assets with complete descriptive information, including asset name, type, maintenance schedule, and system generated asset IDs.

- **Asset Health Monitoring**

PredictiX continuously evaluates the condition of assets by analyzing historical maintenance data

- **Failure Probability Forecasting**

The system predicts the likelihood of asset failure over a future time window, enabling early identification of high risk equipment and supporting preventive maintenance planning.

- **Predictive Maintenance Scheduling**

Based on health trends and failure predictions, the system recommends optimal maintenance dates, reducing unexpected breakdowns and improving equipment availability.

- **Maintenance Ticket Management**

Predictive Maintenance & Smart Ticket Categorization for Asset Management

Users can create and manage maintenance tickets linked to specific assets. Each ticket includes predicted and confirmed priorities, categories, and a detailed lifecycle history from creation to closure.

- **AI-Assisted Ticket Categorization and Prioritization**

The system uses AI models to suggest ticket categories (e.g., electrical, mechanical, software) and priority levels, helping maintenance teams respond faster and more accurately.

- **Automated Ticket Summarization**

PredictiX generates concise summaries for individual tickets and overall maintenance activity, reducing manual reporting effort and improving communication clarity.

- **Warehouse Insight Dashboards**

The system provides interactive dashboards displaying key performance indicators such as average asset health, number of critical assets, ticket volume, and maintenance trends across the warehouse.

- **Report Generation**

Users can generate detailed asset level and warehouse level reports in readable formats. These reports translate analytical results into actionable insights for operational and managerial decision making.

- **AI-Powered Chatbot Interaction**

The chatbot allows users to retrieve asset details, predicted maintenance dates, ticket statuses, and warehouse insights using natural language queries, improving system accessibility.

- **Role Based Access Control**

The system enforces permissions based on user roles, ensuring that administrators and standard users access only the features and data relevant to their responsibilities.

- **Search and Filtering Capabilities**

Users can efficiently search and filter assets and tickets based on multiple criteria such as ID, status, priority, and category, improving navigation and data retrieval.

- **Notifications and Alerts**

PredictiX sends notifications to relevant users when critical assets are detected, tickets are created, or maintenance deadlines approach, enabling timely responses.

Predictive Maintenance & Smart Ticket Categorization for Asset Management

- Cost and Downtime Impact Estimation

The system estimates maintenance costs and potential downtime impacts, helping decision makers evaluate financial and operational risks before failures occur.

2.3 User Classes and Characteristics

- Admin :

Administrators have full control over the PredictiX system and are responsible for overall system management and decision making. An admin can securely log in to the system and manage user accounts by creating, updating, activating, or deactivating users based on organizational requirements. They are responsible for managing warehouse assets, including adding new assets, updating asset information, and monitoring maintenance schedules. Administrators can generate both asset level and warehouse level reports to support strategic planning and operational analysis. In addition, they review predictive maintenance insights such as asset health scores, failure probabilities, and recommended maintenance dates. Warehouse level analytics dashboards enable administrators to track overall asset health, active tickets, risk levels, and maintenance trends across the entire warehouse.

- User :

Standard users access the PredictiX system by logging in with their assigned credentials and primarily focus on operational tasks. Users can submit new maintenance tickets and update existing ones by providing descriptions, comments, and confirmations related to asset issues. They are able to view detailed information about assigned and accessible assets, including health scores, maintenance history, and predicted maintenance dates. Users can monitor the condition of assets relevant to their responsibilities and stay informed about potential risks or upcoming maintenance needs. Additionally, users can review insights such as ticket statuses, recent asset activities, and system generated summaries to support efficient maintenance operations.

2.4 Operating Environment**Hardware Environment:**

PredictiX is designed as a web based system accessible through standard computing devices. The system can be operated on:

- Desktop computers and laptops used in warehouse offices and administrative environments
- Tablets and mobile devices used by maintenance staff for on site asset monitoring and ticket updates

Predictive Maintenance & Smart Ticket Categorization for Asset Management

The system does not require specialized hardware beyond standard devices with internet connectivity. However, if integrated with IoT sensors, compatible sensor hardware may be used to collect asset condition data.

Software Environment:

- Frontend: The frontend of the system is built using Next.js, a Tailwind CSS is used for styling, providing a utility first approach that ensures a responsive and consistent user experience across different devices.
- Backend: The backend services are supported by Supabase, which provides a PostgreSQL based database with built in authentication, real time capabilities, and support for vector embeddings.
- Database: The system uses PostgreSQL as its primary database for storing user data, ticket content, and performance metrics.
- AI Models: PredictiX integrates multiple specialized AI models to support predictive maintenance, cost estimation and intelligent decision making.
- Operating Systems & Versions:

Web Application:

PredictiX is compatible with major operating systems, including:

- Microsoft Windows (Windows 10 and later)
- macOS (recent supported versions)
- Linux distributions commonly used in enterprise environments

The system is accessible through modern web browsers such as Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari, with support for recent stable versions to ensure optimal performance and security.

If deployed in a cloud environment, server side components operate on a Linux based server infrastructure.

Network Environment:

- The system requires a stable internet connection for smooth operation.
- The system uses HTTPS for secure communication between the client and server, ensuring data privacy and security.

Predictive Maintenance & Smart Ticket Categorization for Asset Management

2.5 Design and Implementation Constraints

For the design and implementation of the AI powered predictive maintenance and smart ticket categorization system for warehouse asset management, several constraints guide technology choices and development processes. These constraints ensure the system is scalable, secure, and capable of delivering a high quality user experience while adhering to industry best practices.

- **Cross Platform Compatibility:**

To ensure the system is accessible across a wide range of devices and operating systems, the frontend is built using Next JS, Tailwind and shadcn/ui, for building responsive and interactive user interfaces. This allows the application to run on web browsers and mobile devices without compromising performance or user experience. This allows warehouse staff and administrators to access asset details, maintenance schedules, and dashboards on desktops, tablets, or mobile devices.

- **Privacy and Security:**

The system prioritizes user data security by implementing JWT (JSON Web Tokens) for secure user authentication and session management. All data transmitted between the client and server is encrypted using HTTPS, and sensitive data stored in the database is encrypted. Authenticated access control ensures that only authorized users can access specific functionalities, usernames, ticket list, private data visualizations only for users.

- **Scalability:**

To handle potential growth in the user base and data volume, the backend is designed using FastAPI, a high-performance Python framework known for its scalability and efficiency.

- **Database Flexibility:**

The system uses Supabase for its flexibility and scalability. It is well suited for handling different data types, such as ticket content, user profiles, and performance metrics. Its design allows for easy adaptation to future changes in data structure, ensuring the system can evolve as user needs grow.

- **Advanced AI Features:**

The predictive maintenance module uses machine learning models to forecast potential asset failures based on historical maintenance logs and real time data. This helps reduce downtime by alerting staff before failures occur.

The smart ticket categorization system uses NLP techniques to classify incoming maintenance

Predictive Maintenance & Smart Ticket Categorization for Asset Management

tickets into categories such as Electrical, Mechanical, Software, or Urgent. Auto suggestions are provided, and users validate the predicted category.

- **Real Time Features:**

Real time communication is used to provide live updates on ticket status, predictive alerts, and asset health dashboards. Users can monitor and receive notifications instantly.

- **Content Extraction and Processing:**

The system uses a combination of third party libraries and AI models for content extraction and processing.

The system includes interactive dashboards to display total assets, active vs inactive status, predicted failures, and weighted health scores. Graphs and charts are generated dynamically to show trends for asset types, sensor data, and predicted maintenance events.

- **Export and Integration:**

The system supports exporting reports in PDF format.

2.6 User Documentation

The user documentation for the AI powered predictive maintenance and smart ticket categorization asset management system provides comprehensive guidance to ensure effective utilization of all system features. It includes:

- **User Manual:**

Step by step instructions for functionalities such as asset management , ticket creation, auto-categorization, review, and prioritization, dashboard navigation and graph visualizations. Documentation is provided digitally within the application and as downloadable PDFs. It adheres to industry standards for clarity, accessibility, and user friendliness.

- **FAQs and Troubleshooting:**

A dedicated section addresses common questions related to ticket categorization, predictive alerts, sensor data, and dashboard features.

- **Technical Documentation:**

Backend and AI modules are documented for developers and administrators, including API

Predictive Maintenance & Smart Ticket Categorization for Asset Management

endpoints, database schema, ML model workflow, and integration procedures.

2.7 Assumptions and Dependencies

Assumptions:

- **Availability of Historical Data:**

It is assumed that sufficient historical maintenance logs, repair records, and sensor readings are available to train and validate the predictive maintenance model effectively.

- **Consistent Data Quality:**

The system assumes that maintenance tickets and sensor data are recorded accurately and consistently, without excessive missing or incorrect values that could affect AI model performance.

- **User Participation in Validation:**

It is assumed that users will validate the automatically predicted ticket categories (e.g., Electrical, Mechanical, Urgent), which will help improve model accuracy over time.

- **Stable Internet Connectivity:**

Since the system operates as a web based application, it assumes stable internet access within the warehouse environment for real time updates and dashboard access.

- **Role-Based Usage:**

It is assumed that users will follow defined roles (Admin and Standard User) and access only the features permitted to them under role-based access control policies.

- **Manageable Data Volume in Initial Phase:**

The initial deployment assumes a moderate number of assets and tickets, allowing gradual scaling as warehouse operations expand.

Dependencies:

- **Machine Learning Model Performance:**

The predictive maintenance and ticket classification features depend on the accuracy and

Predictive Maintenance & Smart Ticket Categorization for Asset Management

reliability of the trained machine learning models.

- **Backend Infrastructure :**

The system depends on a stable backend server to handle API requests, database communication, authentication, and AI model integration.

- **Database System :**

Proper storage and retrieval of asset records, maintenance logs, sensor data, and ticket history depend on Supabase's availability and performance.

- **Frontend Framework (NextJS):**

User interaction, dashboard rendering, and real time updates depend on the correct functioning of the NextJS frontend framework.

- **Authentication and Security Mechanisms:**

Secure access to system features depends on JWT based authentication and HTTPS encryption.

- **External Libraries and AI Frameworks:**

The NLP and predictive analytics modules rely on Python based ML libraries, which are essential for training and deploying AI models.

- **Hosting and Deployment Environment:**

The system depends on reliable hosting infrastructure to ensure uptime, scalability, and performance.

3. External Interface Requirements

3.1 User Interfaces

The Predictix system features a modern, AI-powered interface designed to provide a seamless experience for both administrative and standard users. The user interfaces are optimized for clarity, quick navigation, and actionable insights from predictive maintenance data.

List of User Interfaces

- **Login Page:**

Allows users to securely register or log in using email and password credentials. Role selection (Admin or Standard User) is provided during registration.

- **Dashboard:**

Serves as the central hub of the system. Displays KPI cards (Total Assets, Active Assets, Health Score, Prediction Alerts) and visual graphs representing asset performance and maintenance trends. Includes navigation links to All Assets, Tickets, and Predictive Insights.

- **Asset Management Interface:**

Enables CRUD operations on equipment assets:

- Create: Add new assets with details like type, status, and maintenance date.
- Read: View detailed information and health statistics.
- Update: Modify existing asset information.
- Delete: Remove obsolete or decommissioned assets.

Includes filters (ID, Name, Type, Class, Health Score) and time series charts showing sensor data.

- **Single Asset Detail Interface:**

Displays detailed insights into one asset including its maintenance history log, recent sensor readings, and predictive maintenance chart (ML model output).

- **Ticket Creation Interface:**

Allows users to log maintenance issues by entering comments, priority levels, and uploading images or documents. The system suggests a predictive category (e.g: Electrical, Mechanical)

Predictive Maintenance & Smart Ticket Categorization for Asset Management

that is based on description text using AI classification. Users can confirm or correct the suggested category.

- **Ticket Review Interface (Admin/User):**

Shows a list of open and resolved tickets with filtering and sorting options. Includes an Action View for admins to assign, update, or close tickets, add notes, and schedule follow up maintenance through calendar integration.

- **Predictive Insights Page:**

Displays ML based predictions for asset health and failure risk. Includes graphs for Health Trends, Risk Distribution, and Predicted Failures. Users can view asset predictive reports with confidence scores.

- **Chatbot Interface:**

An AI chatbot which responds to queries such as “Show asset ID” or “Predict Asset ID”. Returns current asset status, maintenance recommendation, or predictive model output.

- **User Profile Page:**

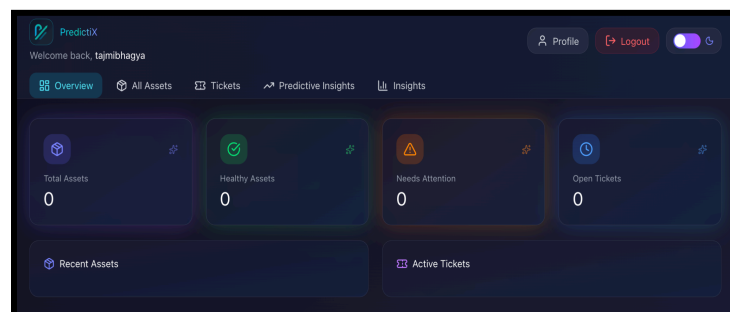
Displays user information such as Name, Role, Email, and Employee ID. Admins can update their profile, change passwords, and manage notification preferences.

- **Users Management Interface:**

Allows administrators to manage system users and control access permissions. Displays a list of registered users with filtering options (Role, Status, Department). Admins can create new users, assign roles (Administrator or User), update user details, reset passwords, activate or deactivate accounts, and monitor basic user activity.

Prototype

Prototypes of these interfaces are attached below (as per screenshots in the design section), illustrating layout, navigation flow, and major components for both Admin and Standard User roles





PredictiX

AI-Powered Predictive Maintenance & Smart Ticket Classification

Login As

User

Admin

Email Address

you@warehouse.com

Password

.....

Sign In

Demo Credentials:

User: user@warehouse.com / password

Admin: admin@warehouse.com / password



PredictiX

AI-Powered Predictive Maintenance & Smart Ticket Classification

Login As

User

Admin

Email Address

you@warehouse.com

Password

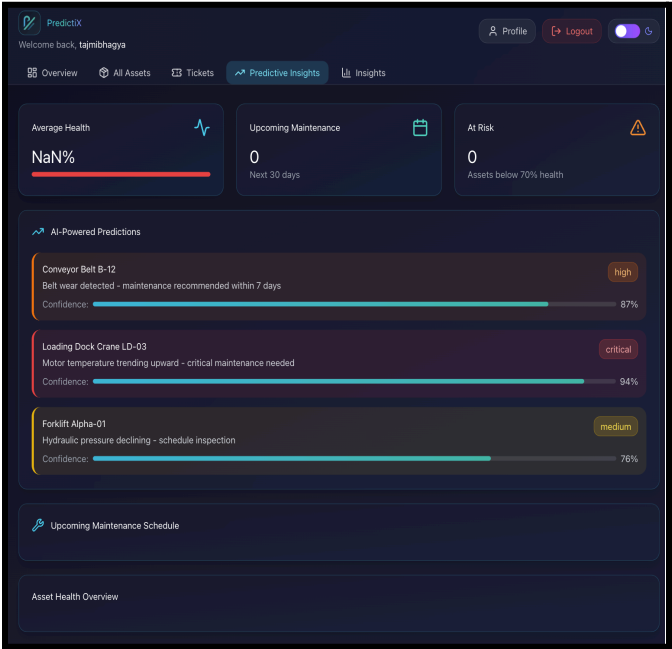
.....

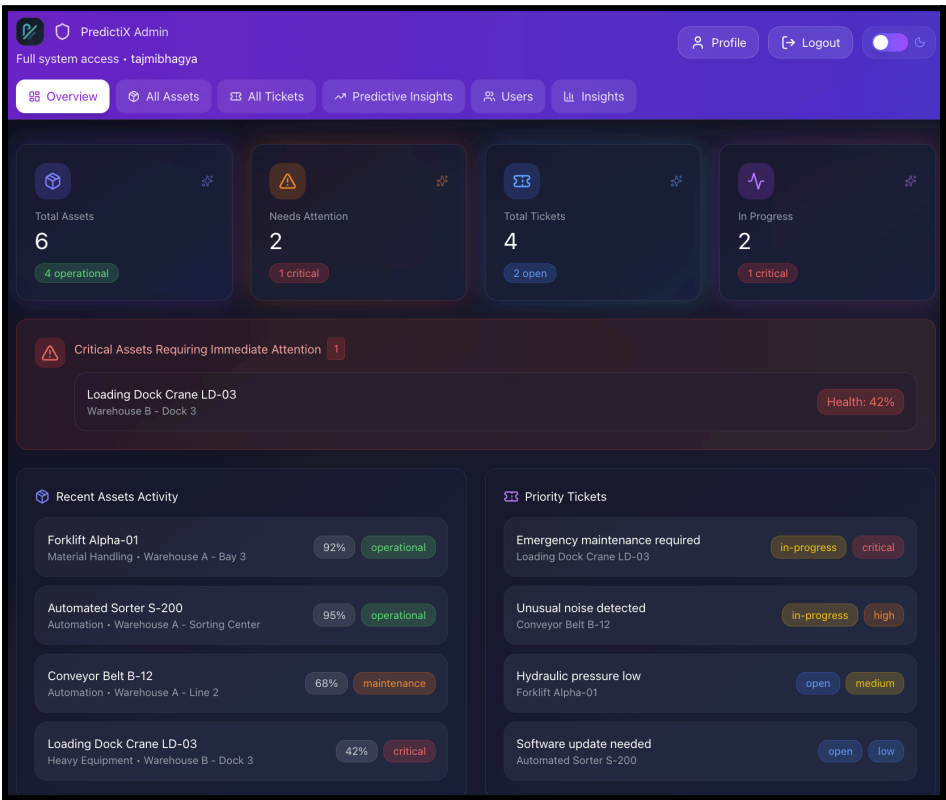
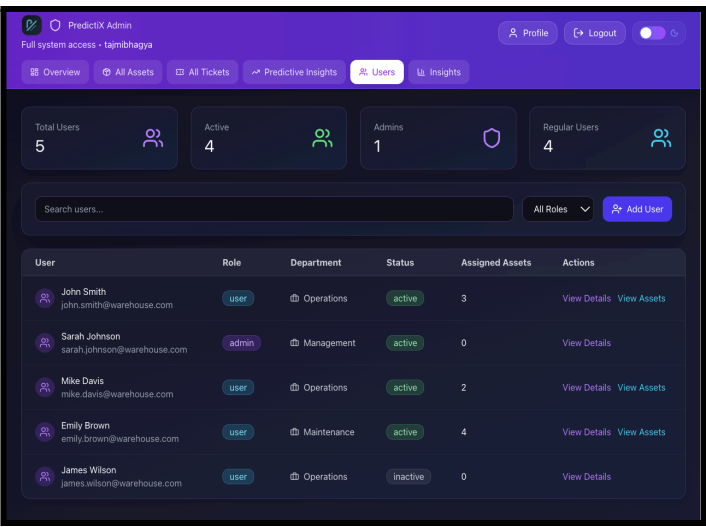
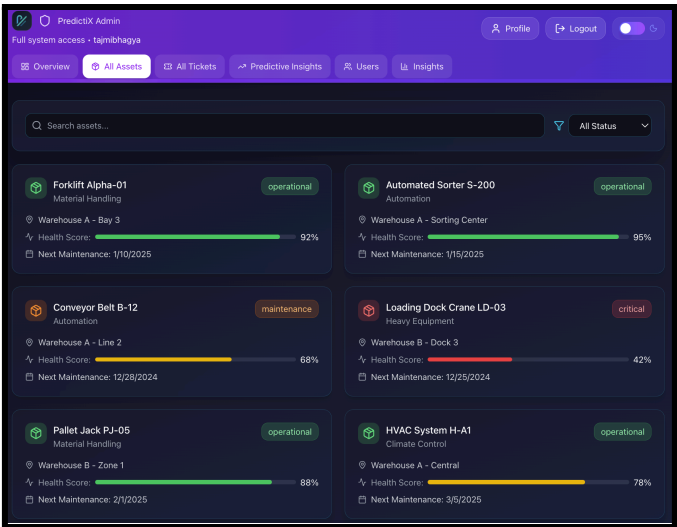
Sign In

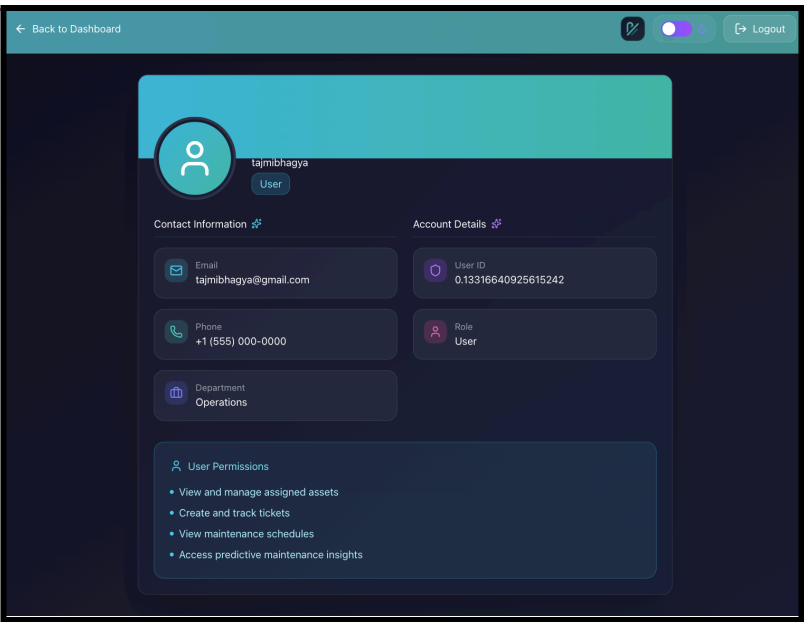
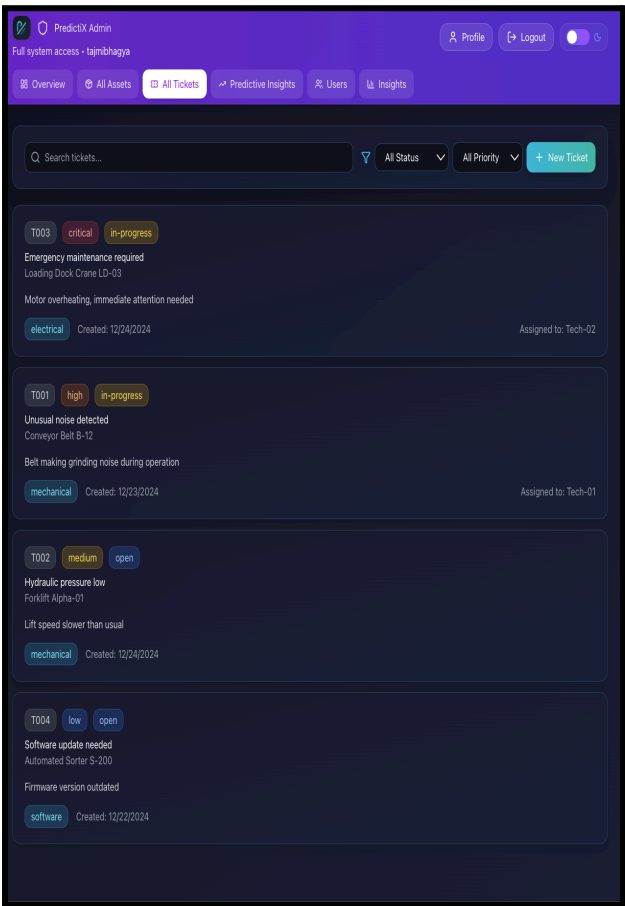
Demo Credentials:

User: user@warehouse.com / password

Admin: admin@warehouse.com / password







3.2 Hardware Interfaces

Input Devices

Predictix is designed to work smoothly with everyday input devices that users already rely on. Administrators and maintenance staff can enter data, upload files, and interact with dashboards easily whether they're at their desks or in the field.

Supported Devices:

- Keyboards and Mice:

Used by desktop and laptop users to navigate through pages, update asset records, and perform CRUD operations with ease.

Display Devices

Since Predictix is a web-based platform, it's built to look great and function smoothly on any modern screen. Whether users are viewing dashboards in an office or checking ticket details in the warehouse, the interface adjusts for comfort and clarity.

Supported Displays:

- Desktop Monitors:

Ideal for admin users who need to view multiple charts or large datasets. The interface works best on monitors with a resolution of 1024x768 or higher.

- Laptops:

The system automatically adapts to different screen sizes, keeping data tables, graphs, and navigation clear and readable.

- Tablets and Smartphones:

Fully responsive and touch friendly, allowing users to interact with prediction results, tickets, and asset details anywhere.

Network Interfaces

Predictix relies on internet connectivity to deliver live dashboards, upload maintenance data, and run predictive AI models.

Supported Networks:

- **Wi-Fi:**

Works seamlessly over standard Wi-Fi networks, ensuring smooth access to cloud hosted dashboards and model predictions.

- **Mobile Data :**

Perfect for technicians who need to check asset health or submit tickets while on site or in remote areas.

3.3 Software Interfaces

Backend Development

The backend of PredictiX is responsible for handling business logic, predictive model execution, data processing, authentication, and communication between system components.

The backend is implemented using FastAPI, a high performance Python framework suitable for API driven architectures and machine learning integration.

Key Interfaces

- **Predictive Maintenance API**

Handles requests between the frontend and the predictive maintenance model.

- Accepts asset data, maintenance logs, and sensor readings.
- Executes machine learning models to compute health scores and failure probabilities.
- Returns prediction results, confidence levels, and recommended maintenance timelines.
- Provides model explainability outputs (e.g., feature contribution insights).

- **Ticket Management API**

Manages maintenance ticket operations and intelligent categorization.

- Processes ticket creation, updates, and status tracking.

- Integrates with the NLP based classification model to suggest ticket categories and priority levels.
 - Stores classification results and allows human confirmation or modification.
 - Report Generation API (RAG Integration)
- Supports AI assisted report generation using Retrieval Augmented Generation (RAG).
 - Retrieves relevant asset and ticket data from structured storage.
 - Queries the vector database for contextual information.
 - Generates grounded analytical summaries using a language model.

- Chatbot API

Handles natural language queries submitted by users.

- Interprets user input.
 - Retrieves contextual warehouse data.
 - Returns structured responses related to asset health, ticket status, cost estimates, or predictive insights.

- Maintenance Cost Estimation API

Calculates projected maintenance and downtime costs.

- Uses historical repair data and predicted failure risk.
 - Computes risk adjusted expected cost values.
 - Returns cost projections and savings comparisons.

- User Management API

Handles authentication and authorization using JWT-based secure token management.

- Manages user registration and login.
 - Assigns and verifies user roles.
 - Controls access to system resources.

- Database Interface

Connects PredictiX to its backend database system.

Performs Create, Read, Update, Delete (CRUD) operations for assets, tickets, maintenance logs, and predictions.

Ensures transactional consistency and secure data access.

Interfaces with both relational storage and vector database components.

Frontend Development

The frontend of PredictiX is developed using NextJS, providing a responsive and interactive web interface.

The frontend communicates with backend APIs to display real time asset information, predictive insights, cost estimations, notifications, and reports.

Key Interfaces

- Dashboard Interface

Fetches data from backend APIs and presents KPIs, predictive alerts, health trends, and maintenance statistics through interactive charts and visual indicators.

- Asset and Ticket Modules

Interact with backend services to:

- Submit and manage maintenance tickets
- Display asset details and prediction results
- Update asset information
- View explainability outputs
- Chatbot Interface

Sends user queries to the backend chatbot API and displays structured responses within the application interface.

Database Management

PredictiX uses Supabase (PostgreSQL) as the primary relational database for structured data storage. A vector database extension (e.g., pgvector) is used to support contextual retrieval for AI based reports and chatbot responses.

Key Interfaces

- **Asset Data Storage**

Stores asset profiles, maintenance history, operational logs, and performance metrics.

- **Ticket Data Storage**

Stores ticket descriptions, comments, attachments, priority levels, and classification outputs.

- **Prediction Data Storage**

Stores model outputs including:

- Health scores
- Failure probabilities
- Confidence levels
- Estimated maintenance dates

- **Cost Estimation Data Storage**

Maintains historical repair costs, downtime durations, and computed economic impact summaries.

AI Model Integration

PredictiX integrates machine learning and AI components within the backend infrastructure.

Key Interfaces

- **Predictive Maintenance Model**

Uses supervised learning models (e.g., XGBoost or regression based models) to estimate asset health and failure risk from historical logs and sensor data.

- **Ticket Classification Model**

Uses NLP techniques to classify maintenance ticket descriptions into predefined categories and priority levels.

- **Explainability Module**

Provides interpretable insights into predictive outputs by identifying key contributing factors influencing model predictions.

- Retrieval Augmented Generation (RAG) Module

Combines structured data retrieval and vector similarity search to generate grounded warehouse level and asset level reports.

3.4 Communication Interfaces

PredictiX relies on secure communication between its frontend, backend, database system and AI modules.

All data transmission is encrypted to ensure confidentiality, integrity, and secure access control.

Communication Protocols

HTTPS

All frontend to backend and backend-to-database communication is conducted over HTTPS to ensure encrypted data transfer.

RESTful API Calls

REST APIs are used for communication between:

- Frontend and backend services
- Backend and AI models
- Backend and external enterprise systems

Data Transfer for Predictive Maintenance

The system exchanges structured asset data, maintenance logs, and sensor readings between the frontend and backend.

Supported formats include:

- JSON (API data exchange)
- CSV (sensor and maintenance log uploads)

Real Time Notifications

When health scores drop below thresholds or high risk predictions are generated, alerts are transmitted to users in real time via backend-triggered notification services.

AI Model Communication

Predictive maintenance models, ticket classification models, cost estimation modules, and RAG based report generation components communicate with the backend using secure RESTful API calls over

Data Export and Reporting

Users may export analytical reports and operational data.

Supported formats include:

- PDF (summary reports)
- CSV / Excel (tabular data export)
- JSON (API-compatible data exchange)

4. System Features

4.1 Feature 1: User Authentication and Role Management

- **Description:**

Ensures secure authentication, session management, and role based access control for administrators and standard users. This feature restricts access to assets, tickets, dashboards, reports, and AI generated insights based on assigned roles.

- **Priority:** High

- **Functional Requirements**

- **REQ-1:** The system shall support secure user registration and login using email and password.
- **REQ-2:** The system shall implement JWT based authentication for secure session handling.
- **REQ-3:** The system shall encrypt user credentials and authentication tokens.
- **REQ-4:** The system shall provide logout functionality and terminate active sessions securely.
- **REQ-5:** The system shall enforce role-based access control (RBAC).
- **REQ-6:** Only administrators shall assign or modify user roles and permissions.
- **REQ-7:** The system shall prevent unauthorized access to restricted modules.
- **REQ-8:** The system shall log authentication attempts for audit purposes.

- **Stimulus / Response Sequences**

- **Stimulus:** User enters login credentials.
Response: System validates credentials and grants or denies access based on role.
- **Stimulus:** User logs out.
Response: System terminates session and redirects to login page.
- **Stimulus:** Administrator assigns or modifies user roles.
Response: System updates permissions and enforces new access rules immediately.

4.2 Feature 2: Asset Management

- **Description**

Enables administrators to manage the full lifecycle of warehouse assets, including operational status, maintenance schedules, and historical records.

- **Priority:** High

- **Functional Requirements**

- **REQ-9:** The system shall allow administrators to create, edit, and delete asset records.
- **REQ-10:** The system shall maintain asset operational status (active, inactive, under maintenance).
- **REQ-11:** The system shall store historical maintenance logs for each asset.
- **REQ-12:** The system shall allow asset search and advanced filtering.
- **REQ-13:** The system shall validate asset data before storage.
- **REQ-14:** The system shall track asset modification history.

- **Stimulus / Response Sequences**

- **Stimulus:** Administrator creates a new asset.
Response: System validates input and stores asset details.
- **Stimulus:** User searches for an asset.
Response: System retrieves and displays matching asset information.
- **Stimulus:** Administrator updates asset details.
Response: System saves modifications and updates asset records.

4.3 Feature 3: Predictive Maintenance and Health Scoring

- **Description**

Applies machine learning models to analyze maintenance logs and sensor data to estimate asset health scores, predict failure probabilities, and recommend maintenance dates.

- **Priority:** High

- **Functional Requirements**

- **REQ-15:** The system shall calculate asset health scores.

- **REQ-16:** The system shall predict asset failure probability.
- **REQ-17:** The system shall estimate recommended maintenance dates.
- **REQ-18:** The system shall display prediction confidence levels.
- **REQ-19:** The system shall apply explainable AI techniques to highlight key influencing factors.
- **REQ-20:** The system shall update predictions when new data is received.
- **Stimulus / Response Sequences**
 - **Stimulus:** System receives updated maintenance logs or sensor data.
Response: Predictive model recalculates health score and failure probability.
 - **Stimulus:** User requests prediction details.
Response: System displays prediction results with explanation and confidence level.

4.4 Feature 4: AI-Based Maintenance Cost Estimation

- **Description**

Estimates potential maintenance and downtime costs using predictive outputs and historical repair data.

- **Priority:** Medium

- **Functional Requirements**

- **REQ-21:** The system shall estimate expected maintenance costs.
- **REQ-22:** The system shall calculate risk adjusted cost using failure probability.
- **REQ-23:** The system shall estimate downtime impact cost.
- **REQ-24:** The system shall display cost range with associated confidence level.
- **REQ-25:** The system shall provide explanation for cost estimation outputs.

- **Stimulus / Response Sequences**

- **Stimulus:** Administrator requests cost prediction for an asset.
Response: System analyzes historical and predictive data and generates cost estimates.

4.5 Feature 5: Maintenance Ticket Management

- **Description**

Allows users to create, categorize, prioritize, summarize, and track maintenance tickets.

- **Priority:** High

- **Functional Requirements**

- **REQ-26:** The system shall allow users to create maintenance tickets.
- **REQ-27:** The system shall classify tickets using NLP techniques.
- **REQ-28:** The system shall predict ticket priority level.
- **REQ-29:** The system shall generate concise ticket summaries.
- **REQ-30:** The system shall maintain ticket history and status tracking.
- **REQ-31:** The system shall allow administrators to assign and close tickets.
- **REQ-32:** The system shall notify relevant users of ticket updates.

- **Stimulus / Response Sequences**

- **Stimulus:** User submits maintenance ticket.
Response: System categorizes ticket and assigns predicted priority.
- **Stimulus:** Administrator updates ticket status.
Response: System records update and send notification.

4.6 Feature 6: Dashboards and Data Visualization

- **Description**

Provides interactive dashboards to visualize asset health, predictions, cost analysis, and ticket statistics.

- **Priority:** Medium

- **Functional Requirements**

- **REQ-33:** The system shall display asset health trends.
- **REQ-34:** The system shall show failure probability forecasts.

- **REQ-35:** The system shall visualize ticket distribution and workload.
- **REQ-36:** The system shall display key performance indicators (KPIs).
- **REQ-37:** The system shall update dashboards dynamically upon data changes.

4.7 Feature 7: AI-Generated Reports (RAG Based)

- **Description**

Generates asset-level and warehouse-level reports using predictive outputs and contextual retrieval.

- **Priority:** Medium

- **Functional Requirements**

- **REQ-38:** The system shall generate structured asset-level reports.
- **REQ-39:** The system shall generate warehouse-level analytical reports.
- **REQ-40:** The system shall ensure reports are grounded in stored data using RAG.
- **REQ-41:** The system shall support report export in PDF format.
- **REQ-42:** Reports shall include visualizations, summaries, and predictive insights.

4.8 Feature 8: AI-Powered Chatbot

- **Description**

Provides natural language interaction for querying asset health, predictions, tickets, and reports.

- **Priority:** Medium

- **Functional Requirements**

- **REQ-43:** The chatbot shall process natural language queries.
- **REQ-44:** The chatbot shall retrieve data from asset, ticket, and prediction databases.
- **REQ-45:** The chatbot shall provide data grounded and explainable responses.
- **REQ-46:** The chatbot shall support follow up contextual queries.

- **Stimulus / Response Sequences**

- **Stimulus:** User asks asset related questions.
Response: Chatbot retrieves relevant system data and generates grounded responses.
- **Stimulus:** User requests explanation of prediction.
Response: Chatbot provides structured and interpretable output.

4.9 Feature 9: Notifications and Alerts

- **Description**

Provides automated alerts for high risk assets, ticket updates, and critical maintenance events.

- **Priority:** High

- **Functional Requirements**

- **REQ-47:** The system shall generate alerts for high failure risk.
- **REQ-48:** The system shall notify users of ticket updates.
- **REQ-49:** The system shall allow users to manage notification preferences.
- **REQ-50:** The system shall log all alert events for traceability.

4.10 Feature 10: Security, Scalability, and Performance

- **Description**

Ensures secure, scalable, and high performance operation suitable for warehouse environments.

- **Priority:** High

- **Functional Requirements**

- **REQ-51:** The system shall use encrypted communication (HTTPS).
- **REQ-52:** The system shall support concurrent users without performance degradation.
- **REQ-53:** The backend shall efficiently process AI computations.
- **REQ-54:** The system architecture shall be scalable for future expansion.

5. Other Non Functional Requirements

5.1 Performance Requirements

- The system shall generate predictive maintenance results within 5 seconds after receiving updated asset logs or sensor data.
- The system shall return chatbot responses within 5 seconds for standard asset-related queries.
- The system shall generate AI based reports within 20 seconds, depending on the size of the selected dataset.
- The system shall support multiple concurrent users without significant performance degradation.
- Dashboard updates and alert notifications shall reflect changes in near real-time.
- The backend shall efficiently process AI computations, ensuring stable performance under expected warehouse data loads.

5.2 Safety Requirements

- The system shall ensure secure storage and processing of asset, maintenance, and user data.
- The system shall use encrypted communication (HTTPS) for all client-server interactions.
- The system shall prevent unauthorized modification of predictive outputs and maintenance records.
- The system shall log critical system actions (e.g., role changes, asset deletion, ticket closure) for audit and accountability.
- The system shall include role-based restrictions to prevent accidental deletion or modification of critical warehouse data.

5.3 Security Requirements

- The system shall use JWT based authentication for secure session management.
- The system shall hash and salt user passwords before storing them in the database.
- The system shall enforce role-based access control (RBAC) across all modules.
- The system shall restrict direct database access to authorized backend services only.
- The system shall protect AI model endpoints from unauthorized access.

- The system shall comply with applicable data protection and privacy regulations where deployed.

5.4 Software Quality Attributes

- Usability
 - The system shall provide a clear and intuitive interface suitable for both technical administrators and maintenance staff.
 - The system shall provide tooltips and contextual explanations for predictive outputs and cost estimations.
- Reliability
 - The system shall ensure high availability during operational warehouse hours.
 - The system shall maintain backup mechanisms to prevent data loss.
- Scalability
 - The system architecture shall support horizontal scaling to accommodate increased asset data and user activity.
 - The database design shall handle growth in maintenance logs and predictive outputs efficiently.
- Maintainability
 - The system shall support modular AI components to allow future model updates or replacements.
 - The backend shall be structured to enable easy maintenance and future feature extensions.
- Accessibility
 - The system shall support a minimum screen resolution of 1024×768 for desktop devices and 720p for mobile devices.
 - The interface shall be responsive across modern browsers.

5.5 Business Rules

- Only authenticated users shall access system dashboards and predictive insights.
- Only administrators shall create, delete, or modify asset records.

- Only authorized users shall create and update maintenance tickets.
- Predictive maintenance outputs shall be advisory in nature and shall not automatically execute maintenance actions without human confirmation.
- Maintenance cost estimations shall be treated as projected values based on historical and predictive data.
- AI generated reports shall be grounded in stored system data and shall not include unsupported external information.
- Notifications for high risk assets shall be generated when predefined risk thresholds are exceeded.
- Role permissions shall determine access to cost estimation, predictive analytics, and report export features.

6. Other Requirements

6.1 Database Requirements

- The system shall store asset data, maintenance records, ticket logs, predictive outputs, and cost estimations in a structured relational database managed via Supabase (PostgreSQL).
- The system shall maintain relationships between assets, maintenance history, tickets, predictions, and users to ensure data consistency and traceability.
- The system shall securely store user credentials using hashed and salted password mechanisms.
- The system shall implement role-based access policies at the database level to restrict unauthorized data access.
- The system shall store AI generated insights, including health scores, failure probabilities, confidence levels, and cost estimations, in structured tables for historical tracking.
- The system shall utilize a vector database extension (e.g., pgvector) to support contextual retrieval for AI-generated reports and chatbot responses.
- The system shall support periodic automated database backups to prevent data loss.

6.2 Internationalization Requirements

- Date formats, number formats, and time zones shall be configurable according to deployment region.

Appendix A: Glossary

API (Application Programming Interface)

Set of protocols and tools .It enable communication between software components

CSS (Cascading Style Sheets)

A stylesheet language used to render the layout and visual appearance of the frontend.

HTML (Hyper Text Markup Language)

The standard language used to create and structure web sites.

ML (Machine Learning)

A field of artificial intelligence that enables system to learn with data and improve predictions without explicit programming.

NLP (Natural Language Processing)

A field of AI .NLP allows computers to understand, and generate human language.

RAG (Retrieval Augmented Generation)

An AI technique use for combine information retrieval with text generation to provide accurate and context aware responses.

SHAP (SHapley Additive Explanations)

A method use for explain machine learning predictions by showing how each feature contributes to the output.

SRS (Software Requirements Specification)

A formal document that describes the functional and non-functional requirements of a software system.

Supabase

An open source platform providing database, authentication, storage and real time services for application development.

UI (User Interface)

The graphical elements through which users interact with the system, including screens, buttons, and forms.

UML (Unified Modeling Language)

A standard modeling language used for visualize system design via diagrams like as use case, sequence, and activity diagrams.

JWT (JSON Web Token)

A secure token based authentication mechanism used to verify user identity and manage session authorization.

PostgreSQL

An open-source relational database management system used to store and manage structured data such as assets, maintenance records, tickets, and predictive outputs in PredictiX.

pgvector

A PostgreSQL extension that enables storage and similarity search of vector embeddings. In PredictiX, pgvector is used to support contextual retrieval for AI generated reports and chatbot responses.

REST API (Representational State Transfer Application Programming Interface)

A web-based communication standard that allows different software components to interact using HTTP requests. In PredictiX, REST APIs enable communication between the frontend, backend, AI models, and database services.

KPI (Key Performance Indicator)

A measurable value used to evaluate system or operational performance. In PredictiX, KPIs include metrics such as average asset health score, high-risk asset count, open ticket volume, and maintenance workload.

RBAC (Role-Based Access Control)

A security mechanism that restricts system access based on assigned user roles. In PredictiX, RBAC ensures that administrators and standard users have access only to functionalities appropriate to their responsibilities.

Appendix B: Analysis Models

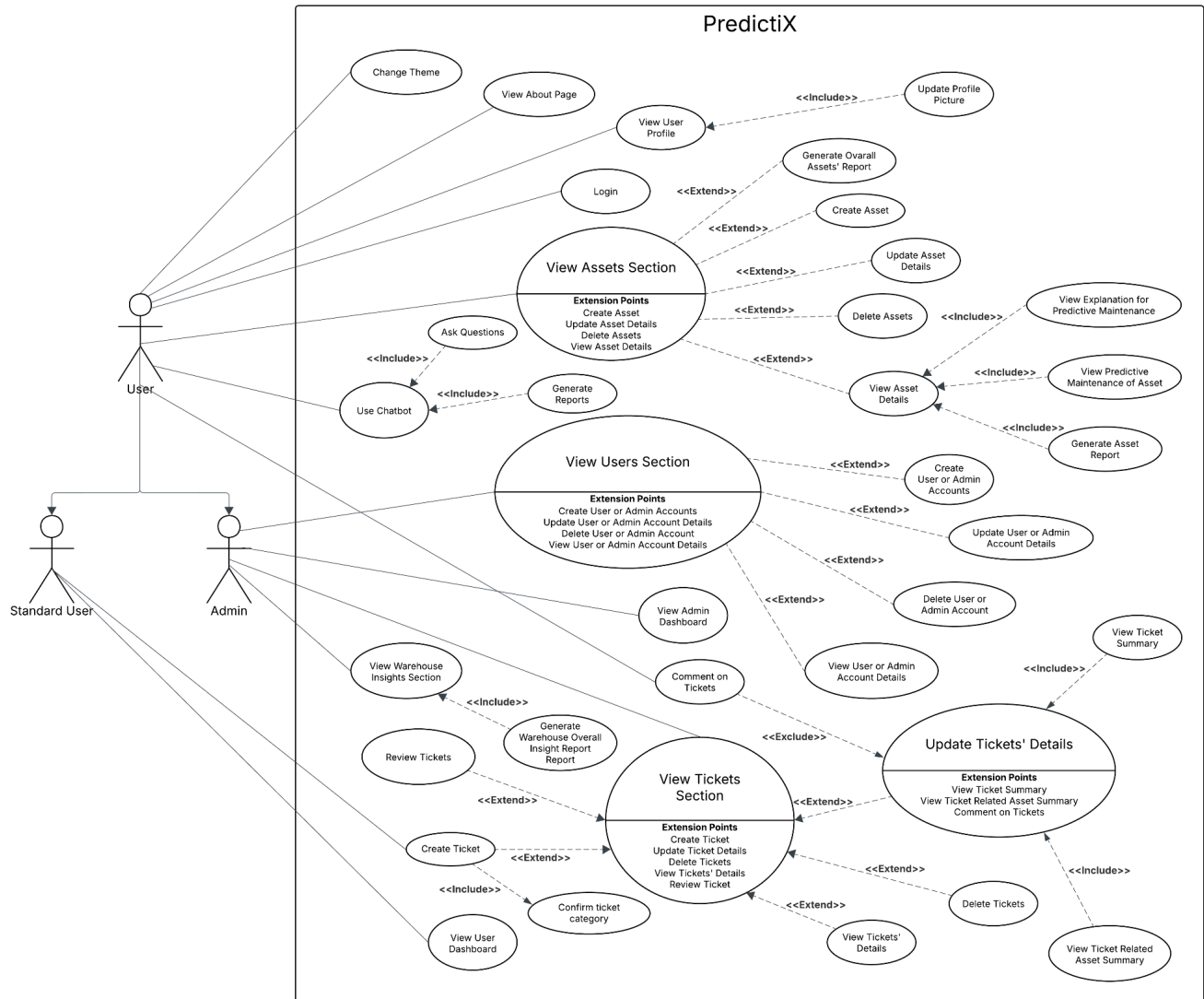


Figure 01: Use Case Diagram

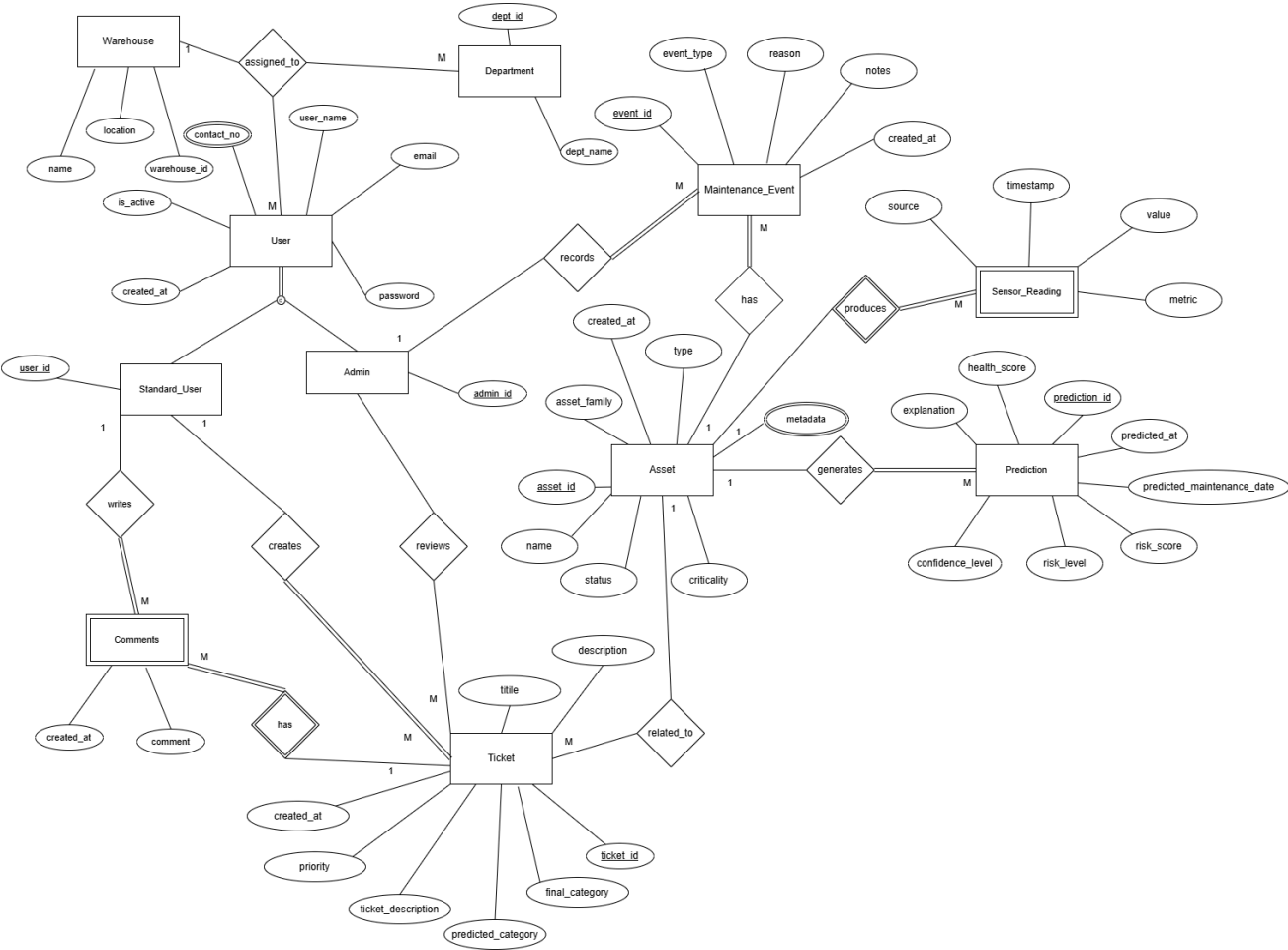


Figure 02: ER Diagram

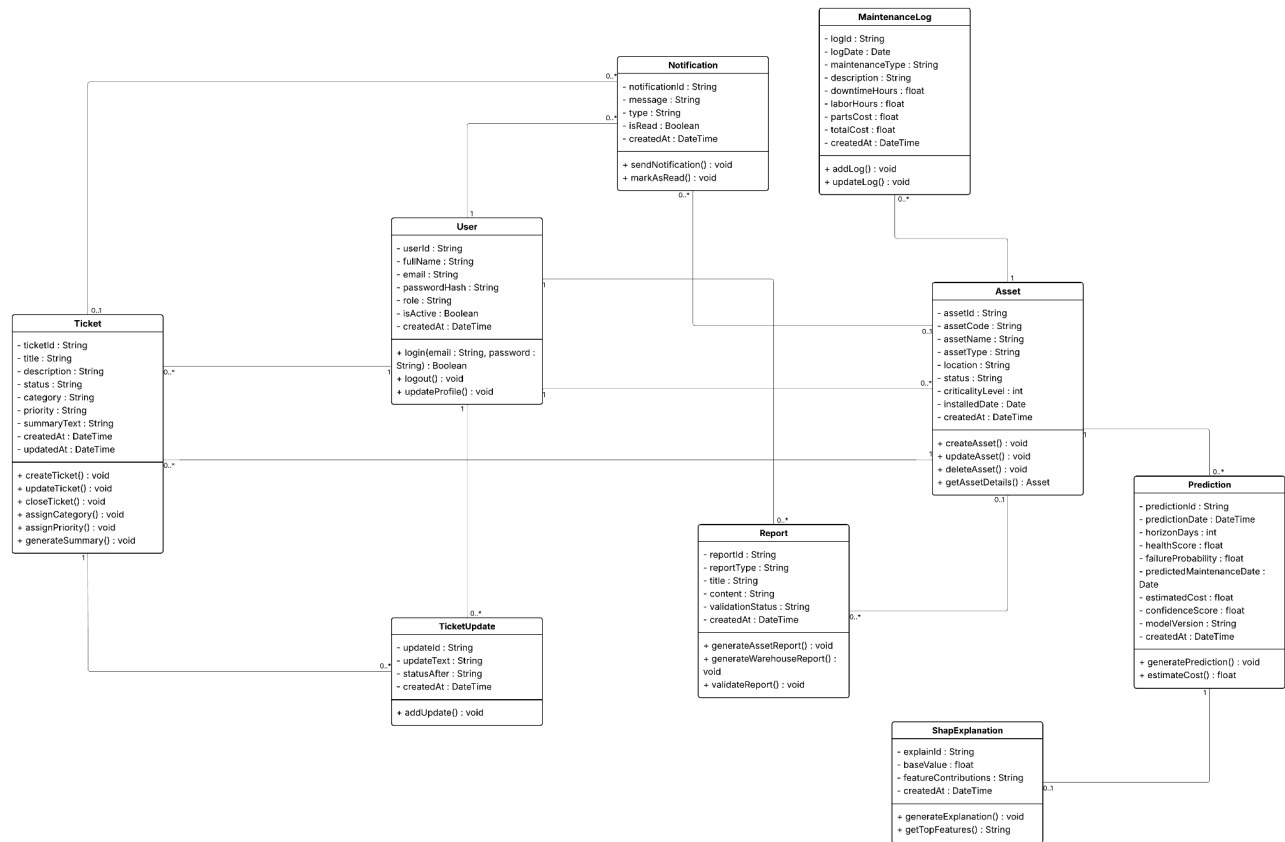


Figure 03: Class Diagram

Appendix C: To Be Determined List

- 1) Determine parameters for predicting asset waste and degradation over time.
- 2) Decide on scalability limits such as maximum number of assets, users, and concurrent system requests.